

Firmware analysis

David Oswald

Code:

```
ODE:0234      call    i2c_read_W_byte    ; Reads W & 0x7F bytes
ODE:0234      ; Init I2C
ODE:0234      ; Set read address to Reg 71 / 70
ODE:0234      ; Stores (inverted) read data at location pointed to by 73 / 72
ODE:0234      ; Sets (75) <- W
ODE:0234      ; Sets bank to 0 (resets bit 5 and 6)
ODE:0234      bsf     BANK0_STATUS, 6
ODE:0235      ; assume bank = 2
ODE:0236      tstf    byte_DATA_119
ODE:0237      skpzn
ODE:0238      ; CODE XREF: maybe_related_to_rewriting_program_memory:endless_
ODE:0238      endless_loop_1:
ODE:0238      b       endless_loop_1
ODE:0239      movlw   2
ODE:023A      call    read_value_from_eeprom ; Address in W
ODE:023A      ; Result in W
ODE:023A      ; Switches bank to 0
ODE:023B      xorlw   OFF
ODE:023C      bsf     BANK2_STATUS, 6
ODE:023D      skpz
ODE:023E
```

Pre-Authentication Remote Code Execution in Netgear SOHO devices

Overview:

A pre-authentication stack overflow vulnerability is present in many Netgear SOHO devices that can allow a network-based attacker to execute code on the device.

Exercising:

```
$ python exploit.py 192.168.1.1
```

The exploit will automatically determine the SOHO device model/version and then exploit it to start telnet on TCP Port 8888. Additional arguments can be used to specify a separate payload command to run or serve the exploit via CSRF.

// Set up passwords

```
add_auth_entry("\x50\x40\x40\x56", "\x5A\x41\x11\x17\x13\x13", 10); // root xc3511
add_auth_entry("\x50\x40\x40\x56", "\x54\x4B\x58\x5A\x54", 9); // root vizxv
add_auth_entry("\x50\x40\x40\x56", "\x43\x46\x4F\x48\x4C", 8); // root admin
add_auth_entry("\x43\x46\x4F\x48\x4C", "\x43\x46\x4F\x48\x4C", 7); // admin admin
add_auth_entry("\x50\x40\x40\x56", "\x1A\x1A\x1A\x1A\x1A\x1A", 6); // root 888888
add_auth_entry("\x50\x40\x40\x56", "\x5A\x4F\x4A\x46\x48\x52\x41", 5); // root xmhdipc
add_auth_entry("\x50\x40\x40\x56", "\x46\x47\x44\x43\x57\x4E\x56", 5); // root default
add_auth_entry("\x50\x40\x40\x56", "\x48\x57\x43\x4C\x56\x47\x41\x4A", 5); // root juantech
add_auth_entry("\x50\x40\x40\x56", "\x13\x10\x11\x16\x17\x14", 5); // root 123456
add_auth_entry("\x50\x40\x40\x56", "\x17\x16\x11\x10\x13", 5); // root 54321
add_auth_entry("\x51\x57\x52\x52\x40\x50\x56", "\x51\x57\x52\x52\x40\x50\x56", 5); // support support
add_auth_entry("\x50\x40\x40\x56", "", 4); // root (root)
```

Goals of this lecture

After this lecture, you will ...

- know the different types of firmware binaries
- understand the main steps of firmware analysis
- be able to use common tools for firmware analysis
- know how to approach unknown firmware binaries

Recap: What is firmware?

- **Wikipedia:** “In computing, firmware is a specific class of computer software that provides the low-level control for a device's specific hardware.”
- Firmware = OS and application logic for embedded devices (sometimes without OS as “bare metal”)
- Virtually all modern embedded devices are programmable with firmware (and not hardwired)
- Embedded systems can contain multiple processors with their own firmware binary each

```
// Set up passwords
add_auth_entry("\x50\x4D\x4D\x56", "\x5A\x41\x11\x17\x13\x13", 10); // root xc3511
add_auth_entry("\x50\x4D\x4D\x56", "\x54\x4B\x58\x5A\x54", 9); // root vizxv
add_auth_entry("\x50\x4D\x4D\x56", "\x43\x46\x4F\x4B\x4C", 8); // root admin
add_auth_entry("\x43\x46\x4F\x4B\x4C", "\x43\x46\x4F\x4B\x4C", 7); // admin admin
add_auth_entry("\x50\x4D\x4D\x56", "\x1A\x1A\x1A\x1A\x1A\x1A", 6); // root 888888
add_auth_entry("\x50\x4D\x4D\x56", "\x5A\x4F\x4A\x46\x4B\x52\x41", 5); // root xmhdipc
add_auth_entry("\x50\x4D\x4D\x56", "\x46\x47\x44\x43\x57\x4E\x56", 5); // root default
add_auth_entry("\x50\x4D\x4D\x56", "\x48\x57\x43\x4C\x56\x47\x41\x4A", 5); // root juantech
add_auth_entry("\x50\x4D\x4D\x56", "\x13\x10\x11\x16\x17\x14", 5); // root 123456
add_auth_entry("\x50\x4D\x4D\x56", "\x17\x16\x11\x10\x13", 5); // root 54321
add_auth_entry("\x51\x57\x52\x52\x4D\x50\x56", "\x51\x57\x52\x52\x4D\x50\x56", 5); // support support
add_auth_entry("\x50\x4D\x4D\x56", "", 4); // root /root/
```

Why firmware analysis?

From Wikipedia, the free encyclopedia

Mirai (Japanese: 未来, *lit.* 'future') is a **malware** that turns networked devices running **Linux** into remotely controlled **bots** that can be used as part of a **botnet** in large-scale network attacks. It primarily targets online consumer devices such as **IP cameras** and **home routers**.^[1] The Mirai botnet was first found in August 2016^[2] by **MalwareMustDie**,^[3] a **white hat** malware research group, and has been used in some of the largest and most disruptive **distributed denial of service (DDoS)** attacks, including an attack on 20 September 2016^[4] on computer security journalist **Brian Krebs**' web site, an attack on French web host **OVH**,^[5] and the **October 2016 Dyn cyberattack**.^{[6][7]} According to a chat log between Anna-senpai and Robert Coelho, Mirai was named after the 2011 TV **anime** series *Mirai Nikki*.^[8]

```
108 // Set up IPv4 header
109 iph->ihl = 5;
110 iph->version = 4;
111 iph->tot_len = htons(sizeof (struct iphdr) + sizeof (struct tcphdr));
112 iph->id = rand_next();
113 iph->ttl = 64;
114 iph->protocol = IPPROTO_TCP;
115
116 // Set up TCP header
117 tcph->dest = htons(23);
118 tcph->source = source_port;
119 tcph->doff = 5;
120 tcph->window = rand_next() & 0xffff;
121 tcph->syn = TRUE;
122
123 // Set up passwords
124 add_auth_entry("\x50\x4D\x4D\x56", "\x5A\x41\x11\x17\x13\x13", 10); // root xc3511
125 add_auth_entry("\x50\x4D\x4D\x56", "\x54\x48\x58\x5A\x54", 9); // root vizxv
126 add_auth_entry("\x50\x4D\x4D\x56", "\x43\x46\x4F\x48\x4C", 8); // root admin
127 add_auth_entry("\x43\x46\x4F\x48\x4C", "\x43\x46\x4F\x48\x4C", 7); // admin admin
128 add_auth_entry("\x50\x4D\x4D\x56", "\x1A\x1A\x1A\x1A\x1A\x1A", 6); // root 888888
129 add_auth_entry("\x50\x4D\x4D\x56", "\x5A\x4F\x4A\x46\x4B\x52\x41", 5); // root xmhdipc
130 add_auth_entry("\x50\x4D\x4D\x56", "\x46\x47\x44\x43\x57\x4E\x56", 5); // root default
131 add_auth_entry("\x50\x4D\x4D\x56", "\x48\x57\x43\x4C\x56\x47\x41\x4A", 5); // root juantech
132 add_auth_entry("\x50\x4D\x4D\x56", "\x13\x10\x11\x16\x17\x14", 5); // root 123456
133 add_auth_entry("\x50\x4D\x4D\x56", "\x17\x16\x11\x10\x13", 5); // root 54321
134 add_auth_entry("\x51\x57\x52\x52\x4D\x50\x56", "\x51\x57\x52\x52\x4D\x50\x56", 5); // support support
135 add_auth_entry("\x50\x4D\x4D\x56", "", 4); // root (none)
136 add_auth_entry("\x43\x46\x4F\x48\x4C", "\x52\x43\x51\x51\x55\x4D\x50\x46", 4); // admin password
```

<https://github.com/jgamblin/Mirai-Source-Code/blob/master/mirai/bot/scanner.c>

Vulnerability overview/description:

Pre-Authentication Remote Code Execution in Netgear SOHO

Sony IPELA ENGINE IP Cameras contain multiple backdoors that, among other functionality, allow an attacker to enable the Telnet/SSH service for remote administration over the network.

Overview:

A pre-authentication stack overflow vulnerability is present in SOHO devices that can allow a network-based attacker to gain access to the device.

Other available functionality may have undesired effects to the camera image quality or other camera functionality.

After enabling Telnet/SSH, another backdoor allows an attacker to gain access to a Linux shell with root privileges!

Exercising:

\$ python exploit.py 192.168.1.1

The exploit will automatically determine the SOHO device IP address and exploit it to start telnet on TCP Port 8888. Additional flags can specify a separate payload command to run or serve the exploit.

The vulnerabilities are exploitable in the default configuration over the network. Exploitation over the Internet is possible, if the web interface of the device is exposed.

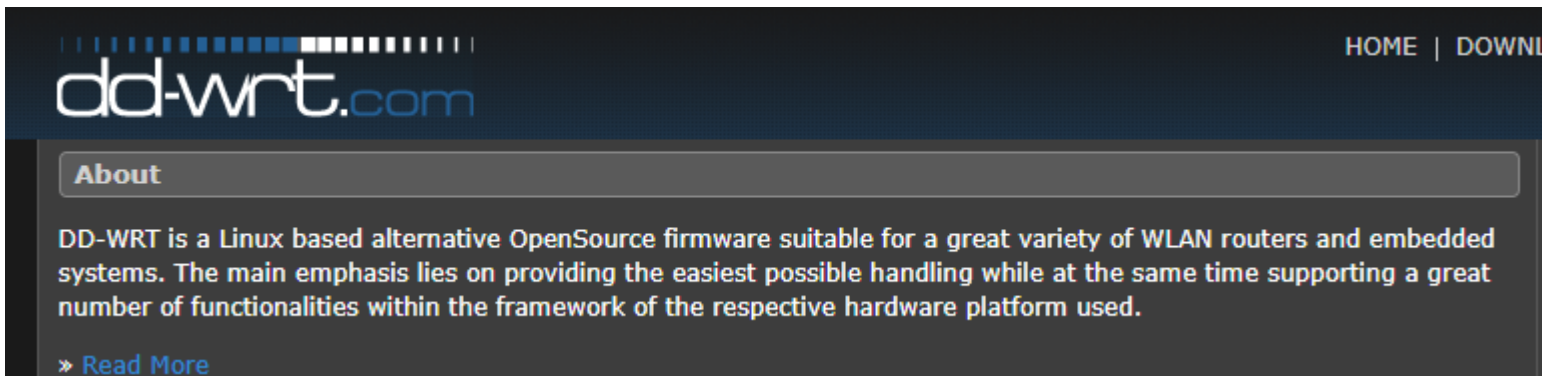
https://sec-consult.com/fixdata/seccons/prod/temedia/advisories_txt/20161206-0_Sony_IPELA_Engine_IP_Cameras_Backdoors_v10.txt

Details:

This vulnerability is a pre-authentication memcpy-based stack overflow in httpd, originally found in the Netgear R7000. The router runs httpd as the main web server that is accessible via the LAN, or if configured, via the WAN. The vulnerability has been present in the R7000 since it was released in 2013 (and earlier for other devices).

Before parsing generic HTTP requests, httpd checks to see if the provided HTTP request is part of the update process. If so, a separate request parser is used, as compared to the generic request parser. If this parser decides that

<https://github.com/grimm-co/NotQuite0DayFriday/blob/master/2020.06.15-netgear/notes.txt>



TUYA-CONVERT

A Chinese company named Tuya offers a free-to-brand turnkey smart home solution to anyone. Using their offer is dead-simple, since everything can be done by clicking through the [Tuya web page](#), from choosing your pre-designed products or pre-programmed wifi-modules (mostly ESP8266) to building your own app. In the end, this has resulted in as they claim over 11 000 devices 'made' by over 10 000 vendors using Tuya's firmware and cloud services.

Aside from that, they claim their cloud solution has 'military grade security'. Michael Steigerwald, founder of the German IT security startup VTRUST, was able to disprove this claim and presented his results in the "Smart home - Smart hack" talk at 35C3 in Leipzig: https://media.ccc.de/v/35c3-9723-smart_home_-_smart_hack

In the following days, VTRUST and the German tech magazine c't decided to work together. Since reflashing devices using the ESP8266/85 is widespread among DIY smart home enthusiasts, we wanted to provide an easy way for everyone to free their devices from the cloud without the need for a soldering iron.

What is #OpenAPS?

The Open Artificial Pancreas System project (#OpenAPS) is an open and transparent effort to make safe and effective basic Artificial Pancreas System (APS) technology widely available to more quickly improve and save as many lives as possible and reduce the burden of Type 1 diabetes.

OpenAPS means basic overnight closed loop APS technology is more widely available to anyone with compatible medical devices who is willing to build their own system.

Black-box analysis

- Similar to pentesting a network, website, ...
- Use standard techniques without firmware
 - Port scans (nmap)
 - Sniffing network traffic (Wireshark)
 - Web vulnerabilities (remember last year)
- Can be sometimes combined with analysis of the actual firmware (aka gray box), or simplify some steps (e.g. checking TCP connections easier than reversing a binary)

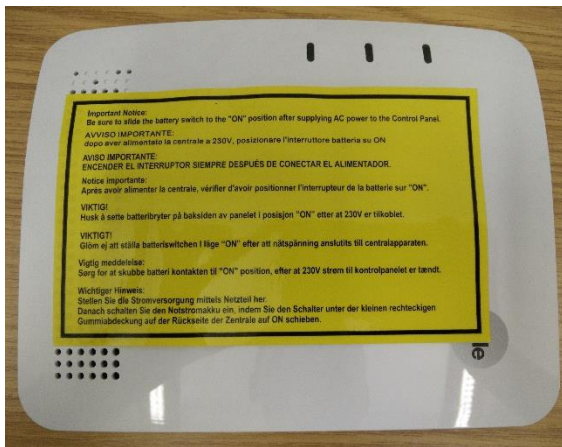
Main steps of white-box firmware analysis

1. Convert dump into appropriate form
2. Identify device & CPU architecture
3. Analyse structure (e.g. partitions)
4. Decrypt (if required)
5. Unpack (if required)
6. If “bare metal”: load into IDA, Ghidra, ...
7. Else: mount filesystem; load into forensics tool (e.g. Autopsy) or use automated analysers
8. Analyse files of interest (e.g. server binaries)

(Some) Tools for firmware analysis

1. **Convert:** objdump, hex editor
2. **Identify:** pictures, cpu_rec, strings
3. **Analyse structure:** binwalk
4. **Decrypt:** usually proprietary
5. **Unpack:** binwalk -e, 7zip, easylzma, cpio, ...
6. **Bare metal rev eng:** IDA, Ghidra
7. **Filesystems:** mount, Autopsy, TSK
8. **Analyse files of interest:** IDA/Ghidra, editor, FACT, ...

Part 1: Bare-metal firmware



Yale alarm

- Firmware extracted via JTAG
- **Identify:** some ARM?
- **Structure/packing:** binwalk?



Live-Demo

“Anything that can go wrong,
will go wrong”

Identifying Yale firmware/hardware

Yale alarm

- Firmware extracted via JTAG
- **Identify:** `chunk(0x54800;169): ARMhf`
→ this likely means ARMv7
- **Structure/packing:**
binwalk: Only finds some HTML/XML strings
strings: finds references to MQX HTTP server —
this is an RTOS
- Let's load it into Ghidra



Live-Demo

“Anything that can go wrong,
will go wrong”

Loading Yale firmware into Ghidra

Part 2: Linux-based firmware



The Hive hub



- Firmware extracted via eMMC
- **Identify:** By looking at the board:
Texas Instruments AM3352BZCE30 (MPU ARM Cortex A8 32-Bit RISC Processor)
- **Structure/packing:** Let's look at Binwalk ...

Hive - binwalk

DECIMAL	HEXADECIMAL	DESCRIPTION

815544	0xC71B8	device tree image (dtb)
...		
2621440	0x280000	uImage header, header size: 64 bytes, header CRC: 0x39A0939E, created: 2014-05-28 13:35:32, image size: 3442423 bytes, Data Address: 0x80008000, Entry Point: 0x80008000, data CRC: 0xE52C82AF, OS: Linux, CPU: ARM, image type: OS Kernel Image, compression type: none, image name: "Linux-3.12.10"
2621504	0x280040	Linux kernel ARM boot executable zImage (little-endian)
...		
7864320	0x780000	uImage header, header size: 64 bytes, header CRC: 0x1DB57753, created: 2014-11-03 14:52:24, image size: 3442415 bytes, Data Address: 0x80008000, Entry Point: 0x80008000, data CRC: 0x133BBEF3, OS: Linux, CPU: ARM, image type: OS Kernel Image, compression type: none, image name: "Linux-3.12.10"
7864384	0x780040	Linux kernel ARM boot executable zImage (little-endian)
7866732	0x78096C	device tree image (dtb)
7881580	0x78436C	device tree image (dtb)
7887152	0x785930	device tree image (dtb)
...		
16023552	0xF48000	Linux EXT filesystem, blocks count: 341808, image size: 350011392, rev 1.0, ext4 filesystem data, UUID=0e786c87-c7fd-485c-a296-2882d9e5d9e5
366039789	0x15D152ED	Zip archive data, at least v1.0 to extract, compressed size: 378, uncompressed size: 750, name: org/apache/commons/net/util/Charsets.class
...		

The Hive hub



- Firmware extracted via eMMC
- **Identify:** By looking at the board:
Texas Instruments AM3352BZCE30 (MPU ARM Cortex A8 32-Bit RISC Processor)
- **Structure/packing:** Let's look at Binwalk ...
 - Seems like a standard Linux image
 - Likely no decryption/unpacking
 - Load it straight into Autopsy (or mount)

Live-Demo

“Anything that can go wrong,
will go wrong”

Analyzing Hive firmware

The Hive hub – some findings



- Many function are just shell scripts
- Older version downloaded initial firmware update via plain HTTP and without signatures ...
- Firmware analysis sometimes also allows firmware modification: we enabled an SSH daemon for remote root access

The Swann smart home hub

- Firmware: see previous lecture
- **Identify**: From boot log:
TI AM335X ES2.1 ARMv7 CPU
- **Structure/packing**: Let's look at Binwalk ...



Swann hub - binwalk

DECIMAL	HEXADECIMAL	DESCRIPTION

5505024	0x540000	JFFS2 filesystem, little endian
5767168	0x580000	ASCII cpio archive (SVR4 with no CRC), file name: "cpiosize.txt", file name length: "0x0000000d", file size: "0x00000010"
5767424	0x580100	Linux kernel ARM boot executable zImage (little-endian)
5783727	0x5840AF	gzip compressed data, maximum compression, from Unix, last modified: 1970-01-01 00:00:00 (null date)
10001864	0x989DC8	device tree image (dtb)
...		
10109060	0x9A4084	ASCII cpio archive (SVR4 with no CRC), file name: "TRAILER!!!", file name length: "0x0000000b", file size: "0x00000000"
12058624	0xB80000	ASCII cpio archive (SVR4 with no CRC), file name: "cpiosize.txt", file name length: "0x0000000d", file size: "0x00000010"
12058880	0xB80100	Linux kernel ARM boot executable zImage (little-endian)
12075183	0xB840AF	gzip compressed data, maximum compression, from Unix, last modified: 1970-01-01 00:00:00 (null date)
16432404	0xFABD14	device tree image (dtb)
...		
16539596	0xFC5FCC	ASCII cpio archive (SVR4 with no CRC), file name: "TRAILER!!!", file name length: "0x0000000b", file size: "0x00000000"
18350080	0x1180000	UBI erase count header, version: 1, EC: 0x2, VID header offset: 0x800, data offset: 0x1000

Live-Demo

“Anything that can go wrong,
will go wrong”

Analyzing Swann firmware

The Swann smart home hub



- Firmware: see previous lecture
- **Identify:** From boot log:
TI AM335X ES2.1 ARMv7 CPU
- **Structure/packing:** Let's look at Binwalk ...
 - Linux kernel image
 - Actual files seem to be UBIFS – not easily mounted in Autopsy or similar
 - Root hash (with strings):
tbKdyTU4E5hZc → sjcwoz3r

[blog.mgrgroup.eu > swann-firmware-update-failed](http://blog.mgrgroup.eu/swann-firmware-update-failed) ▼

[Swann firmware update failed - mgr import](#)

... AM3352BZCZ60 TI AM335x chip; 128M NAND; ZigBee chip CC2538NF23; Username root, password has is **tbKdyTU4E5hZc** which is DES for sjcwoz3r in my ...

Live-Demo

“Anything that can go wrong,
will go wrong”

Swann root shell

Further reading / references

- <https://www.pentestpartners.com/security-blog/how-to-do-firmware-analysis-tools-tips-and-tricks/>
 - <https://embeddedbits.org/reverse-engineering-router-firmware-with-binwalk/>
 - <https://linuxsecurityblog.com/2019/10/03/reverse-engineering-router-firmware/>
 - <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-costin.pdf>
 - <http://www.firmware.re/>
 - https://fk1e-cad.github.io/FACT_core/
 - <https://www.sleuthkit.org/autopsy/>
 - <https://www.j-michel.org/blog/2014/05/27/from-nand-chip-to-files>
- (and many more ...)

Take-home points

- Similar to firmware extraction, firmware analysis is not an “exact science” – every case is different
- Linux-based images can be often mounted and analyzed similar to a “normal” PC
- Bare metal images often hard (*not* impossible)
- Know what you’re looking for, sometimes **strings** is enough

Bonus case study: Amazon Echo



O



on OMAP3

m SD described in
wiki/wiki/Echo

D card w/ modified u-

boot

Drop

- Whole
- laser-

- Conti
- daemons



What about the new Echo?

- Newer Echo devices (dot v2, plus) run FireOS / Android (on MTK8163, similar to Kindle Fire HD8)
- Firmware quite different to old Echo
- Cannot boot from external SD card
- USB interface (adb / bootloader) locked down
<https://medium.com/@micaksica/>
- Dump via eMMC (last lecture)
- Firmware modification?



Echo (MTK) secure boot

1. BROM loader loads and verifies preloader
 2. Preloaders loads and verifies LK (little kernel)
 3. LK loads and verifies Kernel (3.18.19+)
- Step 2 and 3 use custom Amazon certificates, **cannot** modify images (we tried)
 - Step 1 also verifies the preloader in some (undocumented) way, **cannot** modify as well

Modifying system through eMMC

- We still can **read/write** anything
- Certificate injection did not work initially (pinning?)
⇒ try to get root shell to debug
- We started with certain scripts run on boot:
`uid=0 (root) gid=0 (root) context=u:r:init_shell:s0`
- SELinux heavily restricts capabilities
(e.g. cannot access network)
- Can eventually get to adb shell
(and hook the relevant functions)

<https://github.com/tencentbladeteam/Exploit-Amazon-Echo>

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk